Portfolio 1 — Database Design and Modelling

Michael Olayemi Babalola | MIT 8103 Advanced Database Systems | 2026/2027, First Semester

Case study and scope

CampusMart is a simulated Nigerian online retail business serving students and staff. It sells electronics, books, clothing and accessories. Customers place orders, sales staff inspect them, stock staff manage products, and administrators maintain reference data. All records are fictional; prices are in Nigerian naira (NGN).

The database stores customers, categories, products, orders and order items. It supports customer registration, product management, order recording and historical order-value reporting. Payments, shipping, user authentication and refunds are outside the implemented scope.

Relational schema

Relation	Key and principal attributes
CUSTOMERS	PK customer_id; full_name; unique email
CATEGORIES	PK category_id; unique category_name
PRODUCTS	PK product_id; FK category_id; unique sku; product_name; price; stock_quantity
ORDERS	PK order_id; FK customer_id; unique order_reference; order_date; status
ORDER_ITEMS	Composite PK (order_id, product_id); both are FKs; quantity; unit_price

Integrity and normalisation

Required fields use NOT NULL. Foreign keys require valid parents and restrict deletion of referenced records. Checks reject negative stock/prices, nonpositive quantities and unsupported order statuses. Unique constraints identify customer emails, product SKUs, category names and order references.

1NF is supported by storing each order/product combination as a separate row. For 2NF, quantity and charged price depend on the whole composite key in ORDER_ITEMS. For 3NF, customer details stay in CUSTOMERS and category names in CATEGORIES, avoiding transitive duplication. The full functional dependencies and data dictionary are provided with the repository.

Entity relationships and implementation

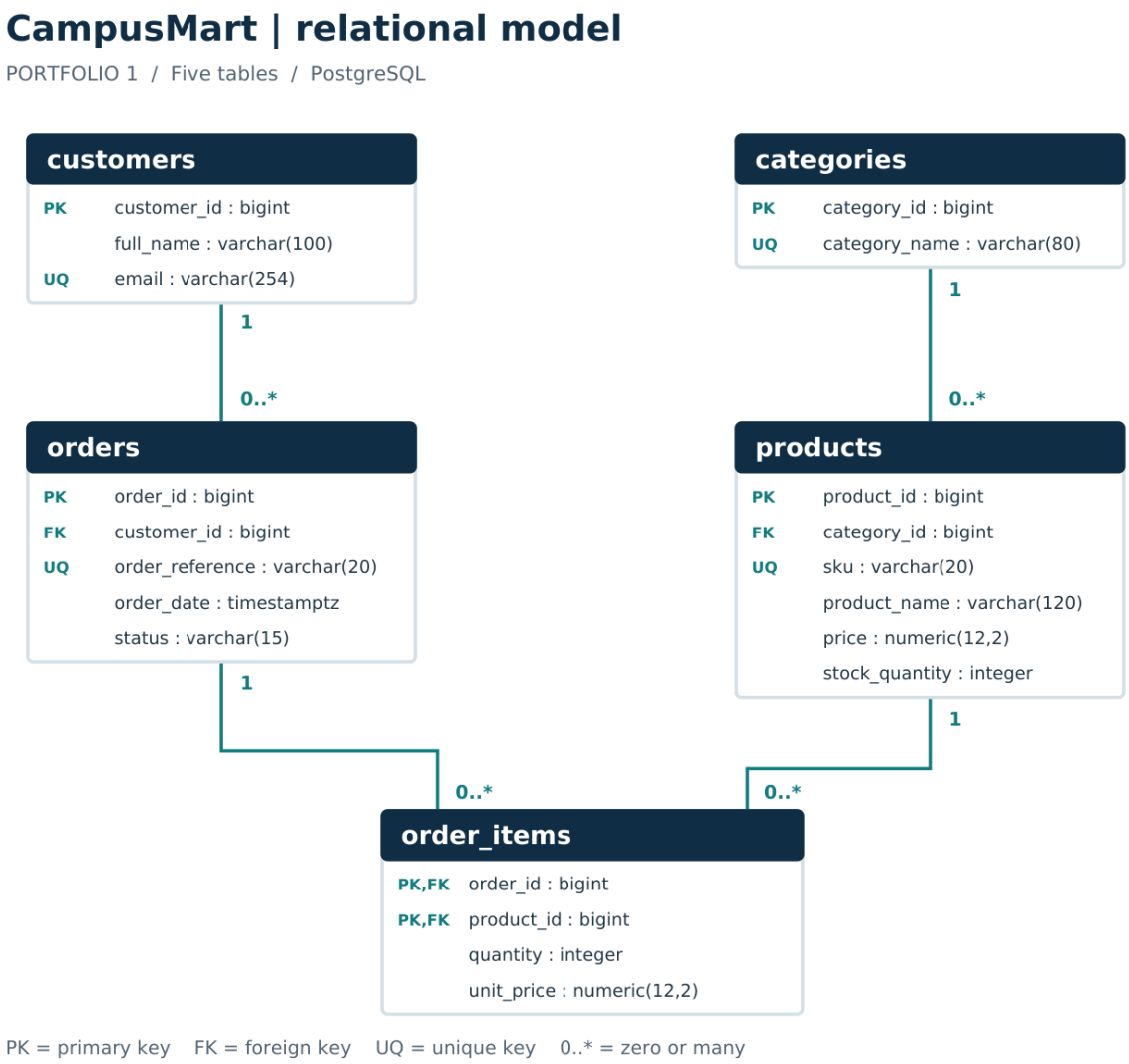


Figure 1. The physical relationships implemented by the SQL. A parent may have zero or many children; each child has exactly one parent on its foreign key. ORDER_ITEMS resolves the many-to-many relationship between orders and products.

The two ORDER_ITEMS identifiers form one primary key. The historical unit_price records what was charged for that order, while PRODUCTS.price records the current catalogue price. These are different business facts.

Implementation sequence

Create a fresh campusmart_portfolio database. Run 01_schema.sql, then 02_seed_data.sql once. Run each numbered SELECT in 03_verify_data.sql and the complete 04_constraint_checks.sql. The scripts are in portfolio-1-database-design/sql/. No script drops existing data.

The intended desktop tools are PostgreSQL and pgAdmin 4. The automated validation recorded for this report used PGlite 0.5.8, which reports embedded PostgreSQL 18.3. The student’s own installed version and desktop screenshots are still awaiting evidence.

Recorded implementation results

The four SQL scripts were executed successfully. The raw JSON report, captured console output and CSV files in evidence/portfolio-1/ preserve the actual results and validation environment.

Table	Actual rows	Result
categories	4	PASS
customers	6	PASS
order_items	16	PASS
orders	8	PASS
products	10	PASS

No.	Invalid operation tested	Result
1	Duplicate customer email	PASS
2	Unknown customer on an order	PASS
3	Negative product stock	PASS
4	Zero order-item quantity	PASS
5	Negative current price	PASS
6	Missing customer name	PASS
7	Duplicate product line in one order	PASS
8	Unknown product category	PASS
9	Delete customer with historical orders	PASS
10	Invalid order status	PASS
11	Negative charged price	PASS
12	Blank product name	PASS

All eight order amounts matched their expected calculations. For CM-1005, the textbook’s charged price was NGN 16,000 and its current catalogue price was NGN 18,000. Its order total was NGN 34,000. This confirms that historical values use charged prices.

The check transaction left permanent row counts unchanged. The repeated-seed guard rejected a second data load. The row-count and constraint outputs are actual automated results, not screenshots from the student’s Windows computer.

Run timestamp: 2026-10-06T20:08:17.987Z

Technical reflection, limitations and sources

Design reflection

Separating current and charged prices protects historical order amounts. The composite order-item key expresses the rule that each product appears once per order, while normalisation reduces repeated customer and category details. These choices make the design easier to explain and maintain.

Validation exposed a version-sensitive detail: restricted parent deletion may report SQLSTATE 23001 or 23503. The check accepts either expected state and still requires the correct constraint name, preventing unrelated errors from being counted as success.

Current limits and remaining evidence

Foreign keys do not require an order to have a child row. Draft orders may be empty; requiring items before placement and atomically updating stock remain workflow tasks for Portfolio 3. A detection query is supplied but does not enforce the workflow. Stock seed values are a current snapshot.

The implementation and its automated results are supplied. A screenshot of the student's own PostgreSQL/pgAdmin execution was mentioned but has not been uploaded for verification or inclusion. Portfolio 1 should not be labelled fully ready for submission until that evidence and the student's review are added. The 2–3 page final LMS summary is prepared after all five portfolios.

AI-use declaration

ChatGPT/Codex assisted with the schema, fictional data, diagram, documentation, automated verification and repository preparation. The assistant executed the supplied scripts in embedded PostgreSQL and recorded the results. The student remains responsible for understanding the work, running it on their installation, and accurately describing their own review, corrections and tests.

References

1. Supplied MIT 8103 Advanced Database Systems Continuous Assessment, 2026/2027: Get hub .pdf.

2. PostgreSQL: Constraints — https://www.postgresql.org/docs/current/ddl-constraints.html

3. pgAdmin: Query Tool — https://www.pgadmin.org/docs/pgadmin4/latest/query_tool.html

4. Electric: PGlite — https://pglite.dev/docs/

Official installation links and additional references are listed in portfolio-1-database-design/references.md. Sources were checked on 6 October 2026.